Faculty of Computers and Artificial Intelligence
Cairo University

Machine Learning Course

Final Project

Material Stream Identification System

Github repo:
https://github.com/Cs-Jimmy/Material-Stream-Classifier/tree/main

Instructors:
Dr. Khaled Wasef
Eng. Ahmed Samir

Name	ID
Laila Mohamed Shawky	20236076
Jumanah Muhammad Ali	20237003
Malak Moustafa Abdel-Maboud	20237015
Basmala Mohsen El-Batran	20237020

Martina Waleed Salah	20237010

Spring 2026

Data Augmentation Pipeline

To enhance the performance and generalization ability of the Machine Learning model, a data augmentation pipeline was implemented. The original dataset contained limited samples and class imbalance between different material categories. Therefore, augmentation techniques were applied to artificially increase dataset diversity and simulate real-world conditions.

The augmentation process helped:

- Reduce overfitting
- Improve model robustness
- Increase dataset diversity
- Enhance classification accuracy on unseen data

Applied Augmentation Techniques

Several augmentation techniques were used during preprocessing:

● Rotation (±12°)

Images were randomly rotated to simulate different object orientations that may occur in real-world environments. This helps the model recognize materials from multiple viewing angles.

● Horizontal Flipping

Images were flipped horizontally to generate additional variations of the same object. This increases feature diversity and helps the model learn orientation-invariant patterns.

● Scaling

Scaling transformations were applied to simulate differences in object size and camera distance. This improves the model's ability to detect materials under varying perspectives.

● Brightness and Contrast Adjustment

Brightness and contrast modifications were used to simulate different lighting conditions. This improves robustness against illumination changes and environmental variations.

● Safe Augmentation Strategy

Augmentation was applied only to the training dataset in order to avoid data leakage between training, validation, and testing sets. Additional safeguards were also implemented to detect duplicate images across dataset splits.

Augmentation Parameters and Purposes

Augmentation	Parameters	Purpose
Horizontal Flip	**p = 0.5**	**Increases orientation diversity**
Random Rotation	**angle in [-12°, +12°]**	**Simulates viewpoint changes**
Scaling / Zoom	**zoom range 0.90–1.00**	**Simulates camera distance**

Brightness & Contrast	**mild adjustment**	**Simulates lighting variation**
Gaussian Noise	**low noise injection**	**Improves robustness**

Dataset Before Augmentation

Before augmentation, the dataset showed class imbalance across material categories. Some classes contained significantly fewer images, which could negatively affect model fairness and classification performance.

1.2 Dataset Balancing

To ensure fair model training and avoid bias toward specific classes, the dataset was balanced by increasing the number of samples in each class to 500 images.

Table 2: Dataset Distribution Before Augmentation

class	Number of Images
cardboard	259
glass	401
metal	328
paper	476
plastic	386
trash	110

To solve the class imbalance problem, additional augmented images were generated for classes with fewer samples until the target count was reached.

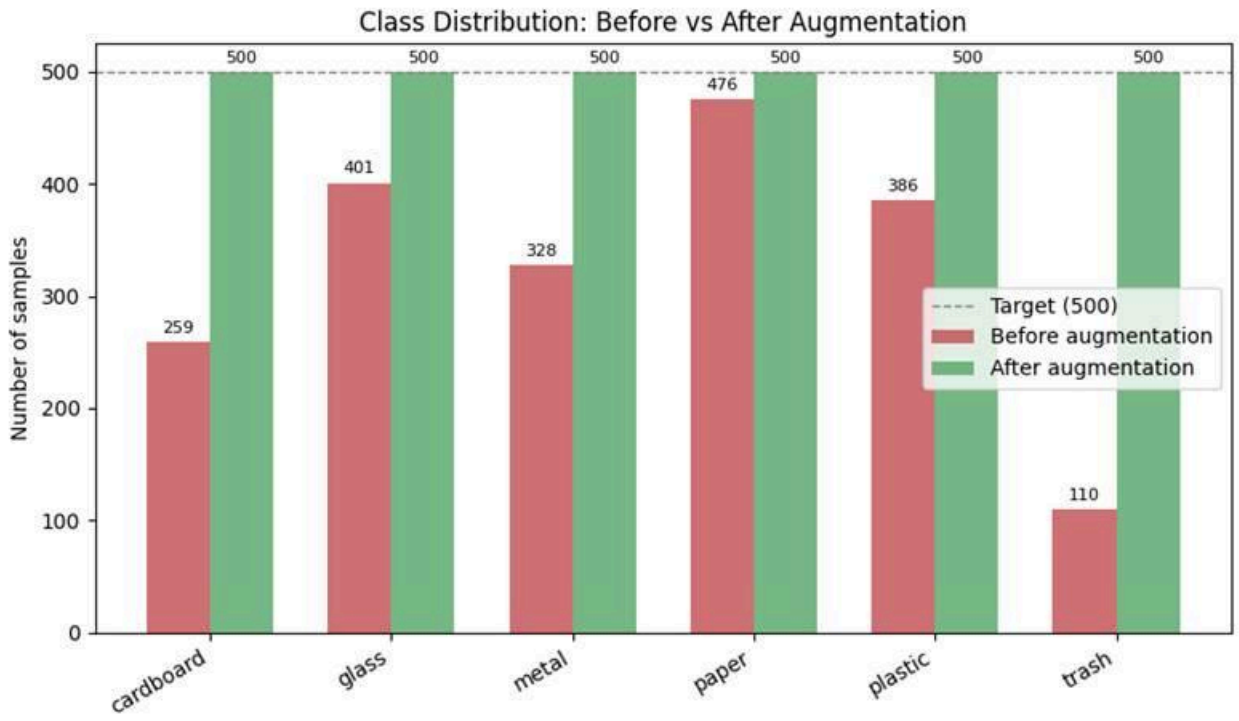
The dataset size increased by more than 30%, satisfying the project augmentation requirement while improving dataset diversity and robustness.

Dataset After Augmentation

After augmentation, each class contained approximately 500 images, resulting in a more balanced dataset suitable for fair model training.

Table 3: Dataset Distribution After Augmentation

Class	Number of Images
Cardboard	500
Glass	500
Metal	500
Paper	500
Plastic	500
Trash	500



The figure compares class distributions before and after augmentation. The augmentation pipeline successfully balanced all material categories to approximately 500 images per class.

Dataset Organization

The final dataset was organized into a structured directory format, where each material category was stored in a separate folder. This organization simplified dataset management and ensured compatibility with downstream Machine Learning processes such as preprocessing, feature extraction, and model training.

augmented_dataset/
  cardboard/
  glass/
  metal/
  paper/
  plastic/
  trash/

This directory structure improved accessibility and allowed the preprocessing pipeline to handle each class independently during augmentation and dataset splitting.

Implementation Details

The data augmentation and preprocessing pipeline was implemented using Python and Jupyter Notebook. The implementation utilized several libraries to support preprocessing, augmentation, visualization, and dataset management.

The following libraries were used:

- OpenCV (cv2) for image preprocessing and augmentation
- NumPy for numerical operations
- Pandas for dataset handling and analysis
- Matplotlib for visualization and class distribution charts
- Scikit-learn for train/validation/test dataset splitting
- Pathlib and OS utilities for file and directory management

The pipeline processes images sequentially, applies safe augmentation techniques, performs leakage and duplicate checking, and saves the augmented outputs directly into structured training directories.

Sample Augmented Image
Example of an augmented image generated by the preprocessing pipeline. The applied transformations include rotation, flipping, scaling, and brightness adjustments to simulate realistic real-world variations.

```
img = cv2.imread(img_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img)
plt.title(f"Augmented Image - {cls}")
plt.axis('off')
```

```
(np.float64(-0.5), np.float64(511.5), np.float64(383.5), np.float64(-0.5))
```

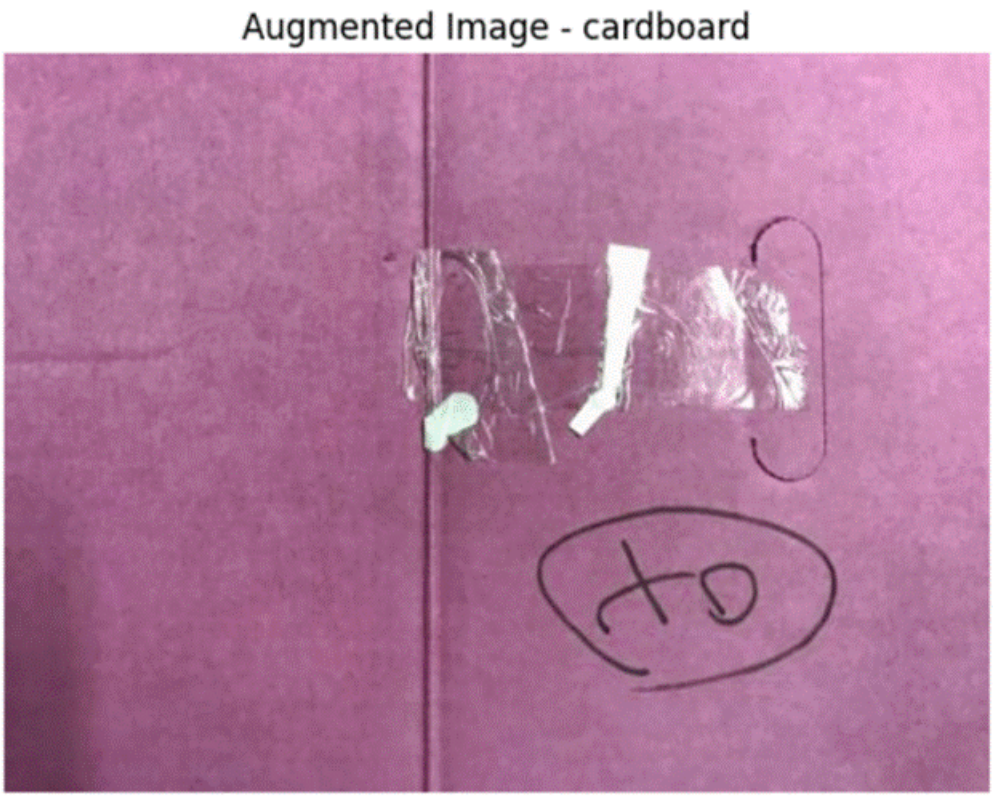


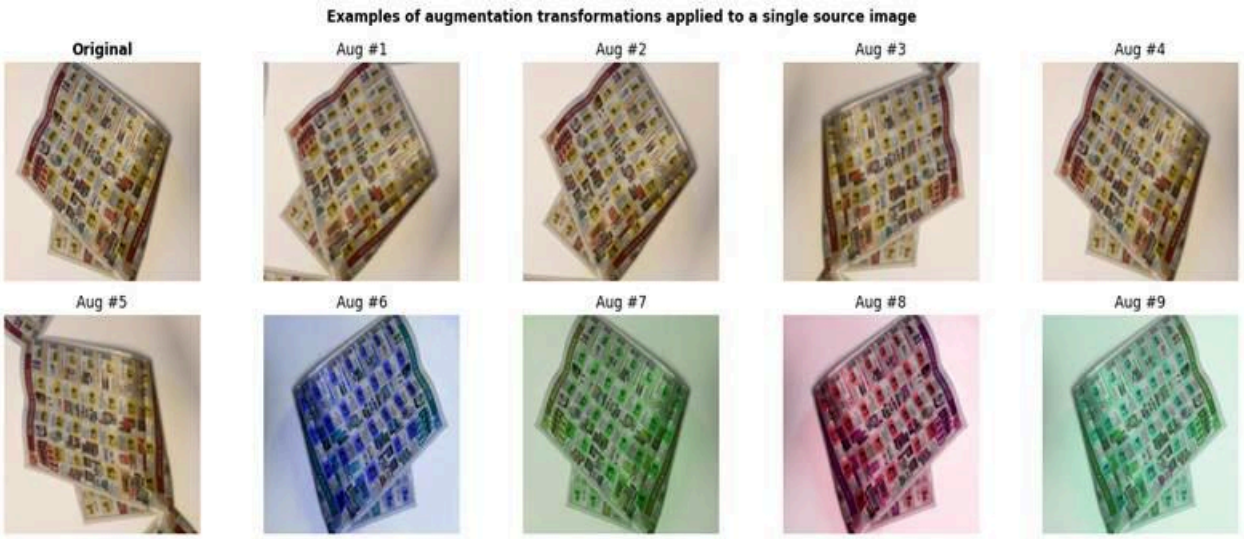
Figure 2: Example of an augmented image generated by the data augmentation pipeline. The applied transformations include rotation, flipping, scaling, and color adjustments, simulating real-world variations.

Validation of Results

The preprocessing and augmentation pipeline was validated by:

- verifying class distributions after augmentation,
- checking duplicate and leakage reports,
- and visually inspecting generated augmented images.

The final dataset achieved a balanced distribution of approximately 500 images per class, improving dataset fairness, training stability, and overall model robustness.



Examples of augmented image variations generated from a single source image. The augmentation pipeline applies transformations such as rotation, flipping, scaling, and color adjustments to increase dataset diversity and simulate realistic real-world conditions.

Section 2: Feature Extraction

Report purpose: This document explains how raw waste-material images are converted into fixed-length numerical features for the classical machine learning classifiers. It also clarifies exactly which feature files and model artefacts are saved by the notebook.

1 Introduction

Classical machine learning classifiers such as Support Vector Machines (SVM) and k-Nearest Neighbours (k-NN) operate on fixed-length numerical vectors. They cannot use raw 2D or 3D image data directly. Feature extraction solves this by converting each image into a compact numerical vector that keeps the useful visual information while reducing unnecessary pixel-level detail.

This section documents the feature extraction methodology used in the MSI system. It explains why HOG and ResNet50 CNN features were selected, how the images are transformed into feature vectors, and how the saved files should be reused later by the SVM, k-NN, and camera application.

2 Descriptor Selection and Justification

Two complementary descriptors were selected. HOG captures the structural edge information of each image, while ResNet50 CNN features capture higher-level visual patterns such as material texture, transparency, and colour-related cues. Combining both descriptors gives a stronger representation than relying on either one alone.

2.1 Histogram of Oriented Gradients (HOG)

HOG computes the distribution of gradient orientations inside small image regions. In this project, it is used to capture object boundaries, shape, and local structural texture, which are useful for distinguishing materials such as cardboard, paper, metal, and plastic.

Implementation parameters used in the notebook:

- **Image size:** 224 x 224 pixels.
- **Cells:** 8 x 8 pixels per cell.
- **Blocks:** 2 x 2 cells per block.
- **Orientation bins:** 9 bins covering 0-180 degrees.
- **Block normalisation:** L2-Hys, which improves robustness to local illumination changes.

Using 224 x 224 for HOG keeps more structural detail than a smaller 128 x 128 input. For this configuration, the HOG sub-vector has 26,244 dimensions.

cardboard - Original

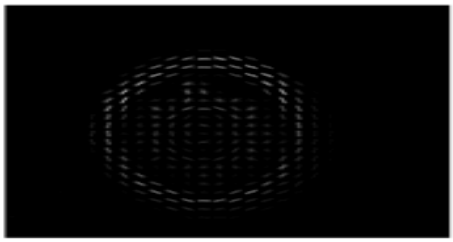
cardboard - HOG

glass - Original

glass - HOG

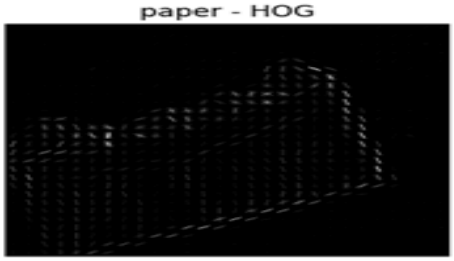
metal - Original

metal - HOG



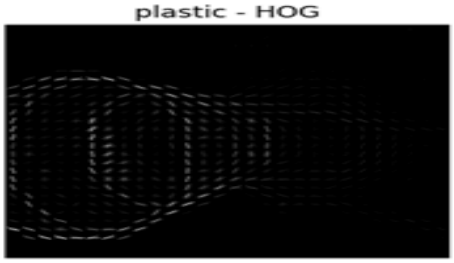
paper - Original

paper - HOG



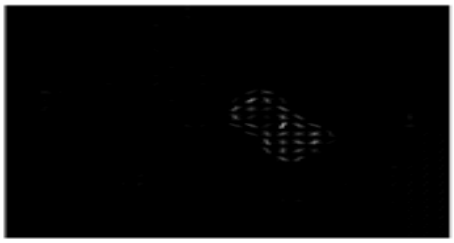
plastic - Original

plastic - HOG



trash - Original

trash - HOG



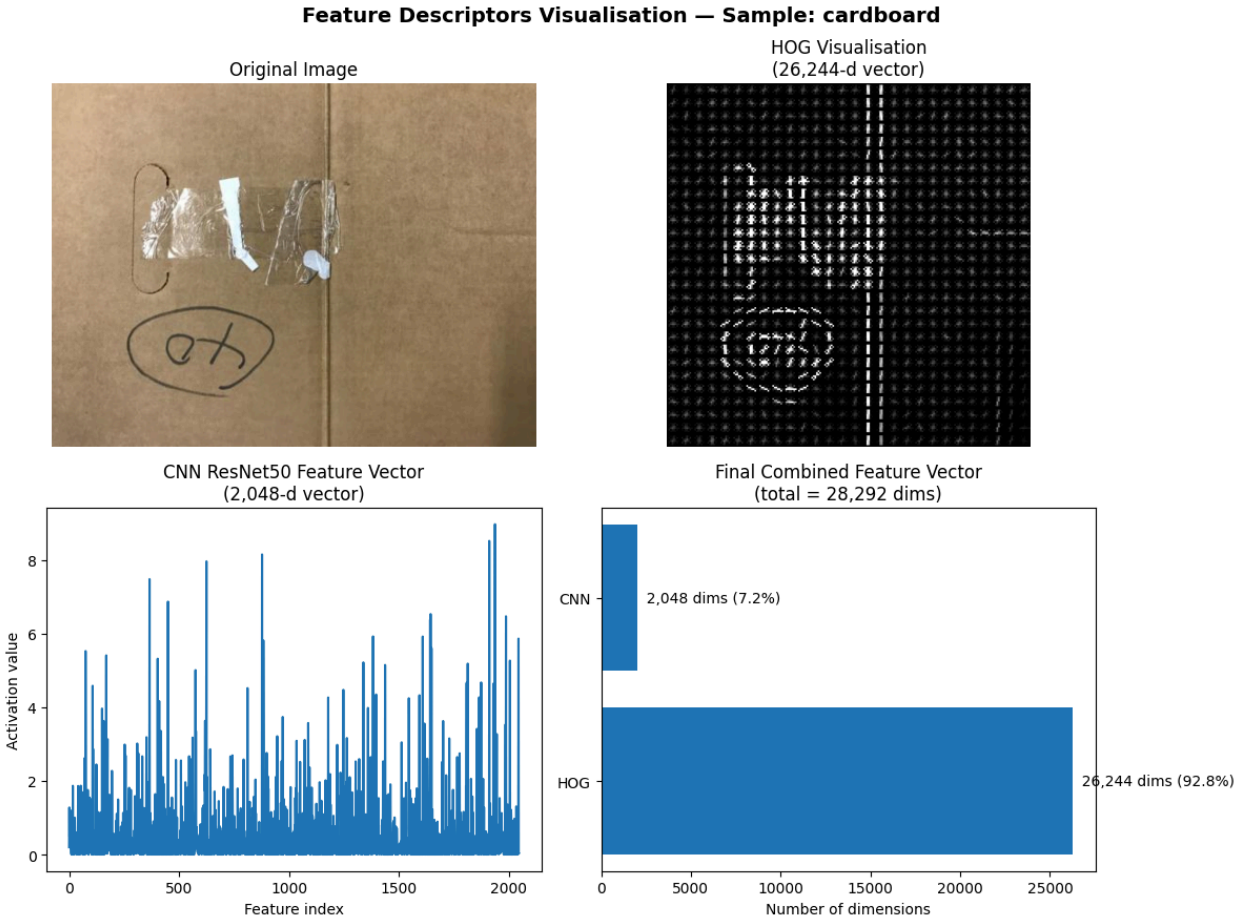
2.2 Deep CNN Features - ResNet50 + GlobalAveragePooling2D

The second descriptor is extracted using a ResNet50 model pre-trained on ImageNet. The classification head is removed and a GlobalAveragePooling2D layer is used to convert the final convolutional feature map into a compact 2,048-dimensional vector.

Design decisions:

- **ResNet50 backbone:** Residual connections allow the model to learn strong visual representations. ImageNet pre-training provides general-purpose visual features that transfer well to waste-material classification.
- **Frozen weights:** No fine-tuning is applied. This reduces overfitting risk and keeps feature extraction deterministic for the classical ML pipeline.
- **GlobalAveragePooling2D:** This reduces the CNN output to 2,048 dimensions instead of flattening a much larger feature map.
- **Input size:** 224 x 224 is used to match the standard ResNet50 image preprocessing setup.

The CNN vector is smaller than the HOG vector, but it carries semantic information learned from a large image dataset. This helps the classifier recognize visual properties that handcrafted gradients alone may not capture.



3. Comparison of Feature Descriptors

The table below summarises why HOG and ResNet50 CNN features were kept, while other common descriptors were not used in the final pipeline.

Table 1: Comparison of selected and rejected feature descriptors

Descriptor	Type	Strengths	Limitations	Decision
HOG	Shape / Edge	Captures gradient orientation and local structure; useful for object boundaries and material shape.	Ignores colour and can be sensitive to large rotations.	SELECTED
CNN ResNet50	Learned / Semantic	Encodes texture, colour-related cues, and high-level visual patterns through ImageNet transfer learning.	Heavier than handcrafted descriptors and less directly interpretable.	SELECTED

LBP	Texture	Fast and useful for micro-texture.	Adds redundancy because the CNN already captures rich texture patterns.	OMITTED
HSV Histogram	Colour	Summarises colour distribution.	Loses spatial layout, and CNN features already provide colour-related information.	OMITTED
SIFT	Keypoint	Scale and rotation invariant; strong for image matching.	Produces a variable number of keypoints, so it is not directly compatible with fixed-length vectors without extra encoding.	REJECTED
Raw Pixels	Global	Simple and requires almost no preprocessing.	Very high-dimensional, sensitive to lighting and scale, and likely to overfit on small datasets.	REJECTED

4. Image-to-Vector Conversion Pipeline

Every image is converted into a single fixed-length vector through the following pipeline. CNN and HOG features are extracted separately, scaled consistently, and then concatenated into one final feature vector.

Table 2: Image-to-vector conversion pipeline

#	Stage	Operation	Output
1	Load image	Read the image from disk using OpenCV. The frame is kept as BGR for HOG processing and prepared for CNN preprocessing when needed.	H x W x 3 image array
2a	CNN resize and preprocessing	Resize to 224 x 224 and apply the ResNet50 preprocess_input function before passing the batch to the feature extractor.	224 x 224 x 3 array
2b	HOG resize and grayscale	Resize to 224 x 224 and convert the image to grayscale before HOG extraction.	224 x 224 grayscale image

3	CNN extraction	Pass batches through the frozen ResNet50 feature extractor with GlobalAveragePooling2D. Batch processing improves memory usage.	2,048-dimensional vector
4	HOG extraction	Compute HOG using 8 x 8 cells, 2 x 2 blocks, 9 orientation bins, and L2-Hys block normalisation.	26,244-dimensional vector
5a	CNN scaling	Apply the saved L2 normalizer consistently to CNN features. Validation, test, and camera frames are transformed only.	2,048-dimensional scaled vector
5b	HOG scaling	Standardise HOG features using the StandardScaler fitted on training HOG features, then apply L2 normalisation.	26,244-dimensional scaled vector
6	Concatenation	Combine the two scaled feature groups in this order: [HOG_scaled | CNN_scaled]. The default weights are hog_weight = 1.0 and cnn_weight = 1.0.	28,292-dimensional final vector

4.1 Final Feature Vector Structure

The final vector length is fixed for every image:

- **HOG sub-vector:** 26,244 dimensions, approximately 92.8% of the total vector length.
- **CNN sub-vector:** 2,048 dimensions, approximately 7.2% of the total vector length.
- **Final vector:** 28,292 dimensions for every image, regardless of the original image resolution.

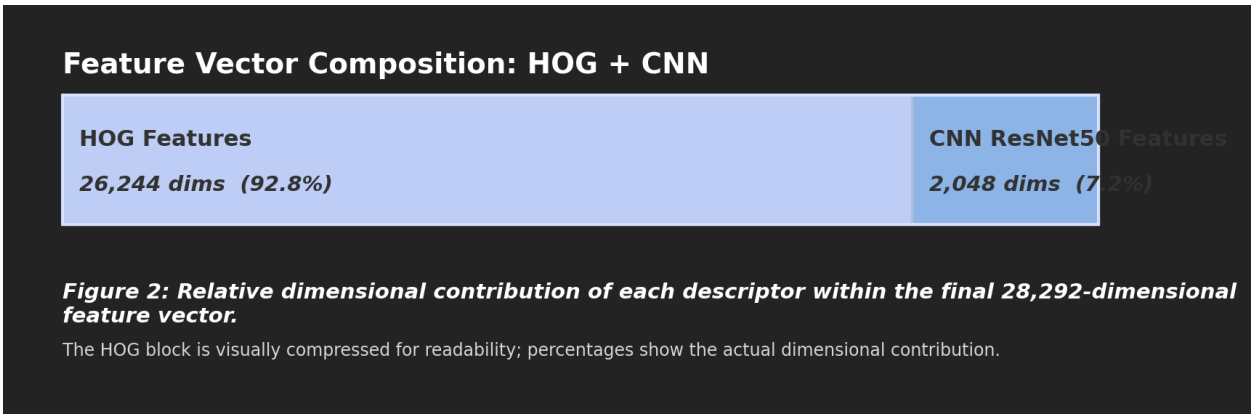


Figure 2: Relative dimensional contribution of each descriptor within the final 28,292-dimensional feature vector.

The HOG block is visually compressed for readability; percentages show the actual dimensional contribution.

Although the HOG part is larger by dimension count, the CNN part still contributes important semantic information. Therefore, both descriptors are useful in the combined representation.

5. Saved Files and Implementation Notes

The notebook does not save separate scaler files for every preprocessing object. Instead, it saves one scaler bundle plus the label encoder. The feature folder contains several .npy files because the notebook saves the main combined features and additional feature sets for comparison or ablation.

Table 3: Saved artefacts produced by the feature extraction notebook

Folder	Files	Purpose
features/	X_train_scaled.npy, X_val_scaled.npy, X_test_scaled.npy	Main final HOG + CNN feature matrices used by the classifiers.
features/	X_train_hog_scaled.npy, X_val_hog_scaled.npy, X_test_hog_scaled.npy	HOG-only features saved to allow comparison or ablation.
features/	X_train_cnn_scaled.npy, X_val_cnn_scaled.npy, X_test_cnn_scaled.npy	CNN-only features saved to allow comparison or ablation.
features/	X_train_hog_cnn_scaled.npy, X_val_hog_cnn_scaled.npy, X_test_hog_cnn_scaled.npy	Explicitly named combined HOG + CNN files, useful for clarity and experiments.
features/	y_train.npy, y_val.npy, y_test.npy and encoded label files	Saved labels for training, validation, and testing.
features/	feature_config.json and hog_cnn_feature_config.json	Configuration files describing the feature extraction setup.
models/	scaler.pkl	One joblib bundle containing cnn_scaler, hog_standard_scaler, hog_normalizer, hog_weight, and cnn_weight.
models/	label_encoder.pkl	Converts predicted numeric labels back to the original class names when needed.

Implementation quality points:

- **No data leakage:** Scalers that learn parameters are fitted on training data only. Validation, test, and camera frames only use transform().
- **Batch processing:** CNN features are extracted in batches to reduce memory usage.
- **Numerical safety:** np.nan_to_num() is used to guard against NaN or Inf values in CNN outputs.
- **Path-overlap check:** The notebook verifies that train, validation, and test image paths do not overlap.
- **Configurable weights:** hog_weight and cnn_weight make it possible to test the contribution of each feature group without rewriting the pipeline.

6. Real-Time Camera Integration Protocol

1. Overview

The real-time camera application is the deployment stage of the MSI pipeline. It integrates the trained SVM classifier into a live webcam feed, classifying materials held in front of the camera and displaying results on screen instantly. The application was implemented in Python using OpenCV for video capture and TensorFlow/Keras for feature extraction.

2. Pipeline

Each frame goes through four stages:

Frame Capture: The webcam feed is captured at 640×480 pixels at 30 FPS using OpenCV's VideoCapture interface.

ROI Extraction: A fixed 250×250 pixel region of interest is cropped from the centre of each frame. A green bounding box guides the user to place the material inside this region. This reduces background interference and lowers the computational cost of feature extraction.

Feature Extraction: The cropped ROI is processed through the same pipeline used during training: resized to 224×224, passed through the frozen ResNet50 backbone with GlobalAveragePooling2D to produce a 2,048-dimensional CNN vector, scaled using the saved cnn_scaler and cnn_weight from scaler.pkl, then reduced with the saved PCA transform (svm_pca_cnn.pkl).

Classification and Rejection: The transformed vector is passed to the SVM. If the maximum class probability is at or above the rejection threshold of 0.5, the predicted class is displayed. If it falls below, the material is labelled Unknown. This is consistent with the rejection strategy used during SVM training.

3. Performance and Interface

To avoid high CPU usage from running ResNet50 on every frame, inference runs on every 10th frame only (approximately 3 predictions per second at 30 FPS). The most recent prediction is held and displayed between inference frames, keeping the video feed smooth.

The overlay displays the predicted class label in colour-coded text, a confidence percentage, and a proportional confidence bar at the bottom of the frame. Each material class has a distinct colour for fast readability. The application exits cleanly on pressing Q.

4. Required Model Files

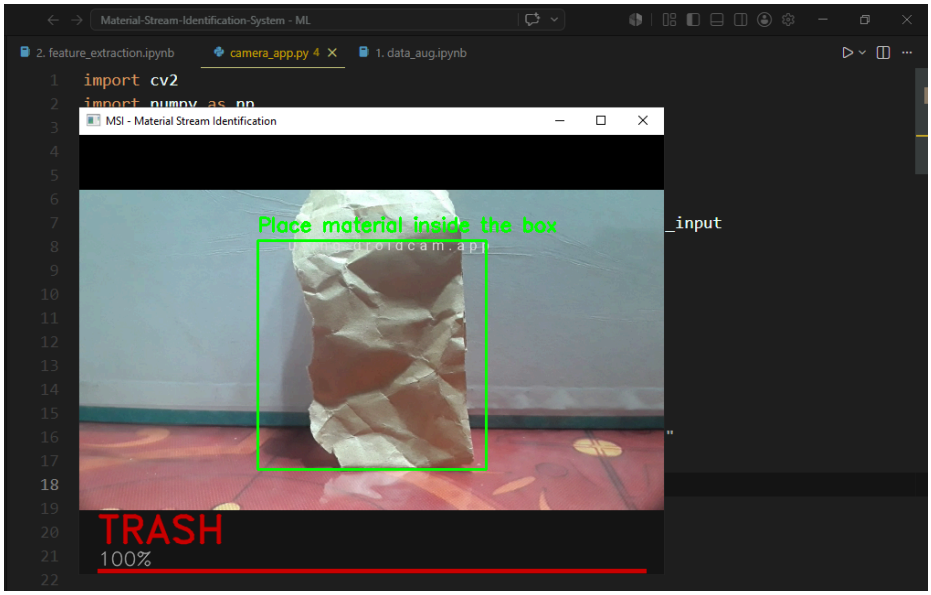
svm_final_model.pkl Trained SVM (RBF, C=10, gamma=scale)

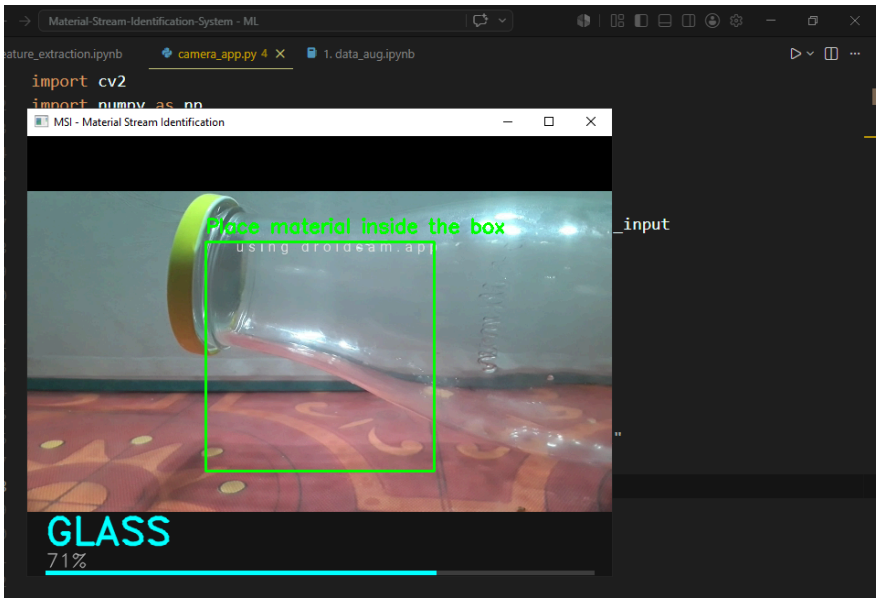
Svm_pca_cnn.pkl PCA fitted on CNN training features

scaler.pkl Bundle containing the CNN scaler and feature weight

label_encoder.pkl Converts numeric predictions to class name strings

Sample tests from Camera app:





7. SVM vs KNN

SVM vs KNN Comparison

Feature Set	SVM		KNN	
	Val Acc	Test Acc	Val Acc	Test Acc
HOG only	0.606	0.601	0.580	0.615
CNN only	0.864	**0.922**	0.867	0.883
HOG + CNN	0.835	0.908	0.885	**0.887**

Rejection Mechanism

Model	Method	Threshold	Accuracy with Rejection
SVM	Max class probability < 0.5 → Unknown	0.5	**91.17%**
KNN	Neighbor vote majority < 0.5 → Unknown	0.5	**90.07%**